

Announcement: "===== GIT Conflit -..."
Reyaz Shaik
Created Feb 2Feb 2

=====
GIT Conflit - Example - latest
=====

```
mkdir git-conflict-demo
cd git-conflict-demo
git init
vi file.txt
"Hello World"
git add file.txt
git commit -m "Initial commit"
```

Create a new branch and switch to it:

```
git checkout -b feature-branch
vi file.txt
Hello from feature branch
git commit -am "Update file.txt in feature-branch"
```

Switch back to the master branch:

```
git checkout master
vi file.txt
Hello from master branch
git commit -am "Update file.txt in main"
```

Try merging feature-branch into master:

```
git merge feature-branch
```

Git detects a conflict:

Automatic merge failed; fix conflicts and then commit the result.

◆ Step 4: Resolve the Conflict

Open file.txt, and you will see:

```
<<<<<<< HEAD
Hello from main branch
=====
Hello from feature branch
>>>>>>> feature-branch
```

Manually edit the file to keep the correct version:

```
git add file.txt
```

```
git commit -m "Resolved merge conflict"
```

◆ Step 5: Verify the Merge

Check the commit history to confirm the conflict resolution:

```
git log --oneline --graph --all
```

=====

GIT FORK - same as clone (remote to local) but fork (download code from one GitHub account to another GitHub account)

=====

= If i want to enhance the code of another guy who has his own repo and code, instead me doing changes in his account directly i will fork it in my GitHub repo and do changes

In GitHub browser - search for another GitHub account(<https://github.com/ReyazShaik/test>) and fork to my account(trainerreyaz account)

GIT FORK and GIT CLONE -- Both repos should be public, for private use token

you cannot unfork, you need to delete the repo

PR - PULL REQUEST

Be in trainerreyaz account and add or modify python1 file and commit

--> contribute --> pull request --> Finish

--> Go to actual reyazshaik GitHub account --> on top --> pull requests --> Click on Request and Merge

-- Again go to trainerreyaz account modify the file --> save --> contribute --> Create pull request

-- Come to reyazshaik account --> pull requests --> see what exactly code change is --> click on commits --> REview changes --> Add comment --> i dont like --> close the request

Squash and merge : combining the commits history

Use squash and merge for branches with many small or work-in-progress commits to keep the history clean.

Squash and Merge is a method used during the pull request (PR) or branch merging process to combine all the commits from a feature branch into a single commit on the target branch. This is particularly useful for maintaining a clean and concise commit history.

Rebase and Merge : Retains individual commits

=====

PR - Pull Request

=====

Everything what we did on git, now its do on GitHub

merge, rebase etc we did on git(local),

TO merge in github use PR

MERGE ON GITHUB - PR should be done on GitHub - from one branch to another branch

create a repo in GitHub and add a file in master branch...

create another branch test and add a file

Now create a PR

In GitHub on top --> Pull request --> New Pull Request --> select branches from where to where (test to master) --> see differences --> Create Pull request --> pull request --> Merge pull request and confirm

Now code merges from branch to master

=====

Rename the Branch Name - GIT

=====

touch python{1..5}

git add .

git commit -m "python" .

git branch dev1branch

git branch -m dev1branch testbranch

git checkout testbranch

touch java{1..5}

git add .

git commit -m "java" .

create a repo and push the files and branch

git remote add origin https://github.com/ReyazShaik/branching.git

git push origin master

git push origin testbranch

create a pull request and merge in GitHub

=====

GIT REVERT in GITHUB

=====

select View all branches in branch section --> which branch you want to revert --> select that branch --> Click PR and revert --> it will create a revertbranch and revert

=====

git clean : delete untrack files

create a file and dont track

git clean -n filename -- first git will ask

git clean -f filename -- delete without asking

=====

CHERRY PICK - Merging the specific files based on commits

=====

mkdir Reyaz

cd Reyaz

git init

touch file1

git add file1

git commit -m "file1" file1

git branch

git branch -m master branch1 [Rename master branch to branch1 for clarity]

git checkout -b branch2 [create branch2]

git checkout branch1 [be in branch1]

touch java{1..5}

git add java*

git commit -m "javafiles" java*

touch python{1..5}

git add python*

git commit -m "pythonfiles" python*

touch php{1..5}

git add php*

git commit -m "phpfiles" php*

to see how many commits

git log --oneline

ls

git checkout branch2 [switch to branch2, in this branch we have only one file]

i want to merge branch2 with branch1 we use git merge, but i dont want all files, i need only few [cherry pick]

git cherry-pick commit-id

git cherry-pick 16dufg(pythonfiles)

git cherry-pick 13gh56g/phpfiles)

ls

Now, this merge only pythonfiles and phpfiles

if you want to revert -- git revert commit-id

GIT TAG - For reference

=====

git log --oneline

git tag tagname commitid

example: git tag dev/prod 0620179

git log --oneline

=====

GIT IGNORE - if you don't want to track the file, add filename in .gitignore

=====

In Same directory

touch file1.txt

touch file2.txt

git status

-- If I don't want to track

vi .gitignore

file1.txt

-- git status --- cant see file1

=====

Delete Branch in GIT

=====

- git branch -D branchname

To get Branch restore - git checkout branchname

git branch

Restore deleted branch in GIT

=====

git reflog

git checkout -b dev 3e98e1c(commitid)

Be in deleted branch and ls

=====

Host website on GitHub

=====

Create a repo and create index.html --> put some html code

setting --> pages --> branch master --> /root --> save --> you get the link

=====

MAKE REPO Public to Private in GitHub --> Repo settings --> danger zone -->

change visibility

=====

=====

Delete branch in GitHub

=====

view branches --> delete icon -- undo there itself -- rename the branch also there